# CHAPTER 4

# CAPSTONE PROTOTYPE DOCUMENTATION

## 4.1 ABSTRACT

With the rapid growth of assistive technologies, the demand for affordable and reliable communication aids for persons with speech and hearing impairments has become increasingly significant. Conventional communication tools are either too expensive, require professional assistance, or lack real-time responsiveness. To address these limitations, this project proposes a Smart Glove for Sign-to-Speech Communication, a wearable assistive device capable of translating predefined sign language gestures into spoken words in real time.

The proposed system employs a set of flex sensors embedded within a glove to capture the bending angles of the wearer's fingers during sign language gestures. These analog signals are processed by an ESP32 microcontroller, which classifies the gesture against a predefined set of recognized signs. Upon recognition, the ESP32 triggers a DFPlayer Mini audio module to play a corresponding pre-recorded voice message through a speaker, effectively converting the detected hand gesture into audible speech.

The system integrates the following key components:

- Flex sensors for finger-bending detection
- ESP32 microcontroller for signal processing and gesture recognition
- DFPlayer Mini audio module for voice playback
- Speaker for audible output
- Rechargeable battery for portable, untethered operation

When a recognized sign gesture is performed, the system identifies the gesture from sensor readings and plays the matching pre-recorded audio message within milliseconds. Unrecognized gesturesare safely ignored until a valid pattern is detected.

This system is low-cost, compact, battery-operated, and user-friendly, enabling deployment in homes, educational institutions, healthcare facilities, and public environments without the need for a trained sign language interpreter. By providing real-time, independent communication support, the Smart Glove has the potential to enhance social participation, reduce communication barriers, and contribute to greater inclusion of individuals with speech and hearing impairments.

## 4.2 INTRODUCTION

Communication barriers severely limit speech- and hearing-impaired individuals in daily communication. Additionally, the lack of affordable wearable translation devices hinders social inclusion.To address these challenges, this project develops a Smart Glove for Sign-to-Speech Communication using embedded technology. The ESP32 microcontroller is selected due to its:

- Built-in ADC for flex sensor reading
- High processing speed
- Low power consumption
- Compact size for wearable design

By combining gesture recognition and audio playback in a single system, the project ensures:

- Real-time sign-to-speech conversion
- Portable, independent communication
- Low-cost assistive technology.

## 4.3 OBJECTIVE OF THE WORK

The objectives of this project are:

- Detect sign gestures using flex sensors on a smart glove.
- Process gestures in real-time with an ESP32 microcontroller.
- Convert gestures to speech using a DFPlayer Mini module.
- Enable independent communication for speech-impaired users.
- Design a low-cost, portable, and user-friendly communication aid.

## 4.4 SYSTEM ARCHITECTURE

The system architecture consists of :

- Input Devices.
- Processing Unit..
- Output Devices.

### INPUTS :

- Flex sensors (4 nos. on glove fingers).

### PROCESSING UNIT :

- ESP32 microcontroller.

### OUTPUTS :

- DFPlayer Mini audio module .
- Speaker (0.5W, 8Ω) .
- Mobile app notification.

### 4.1.1PROPOSED BLOCK DIAGRAM

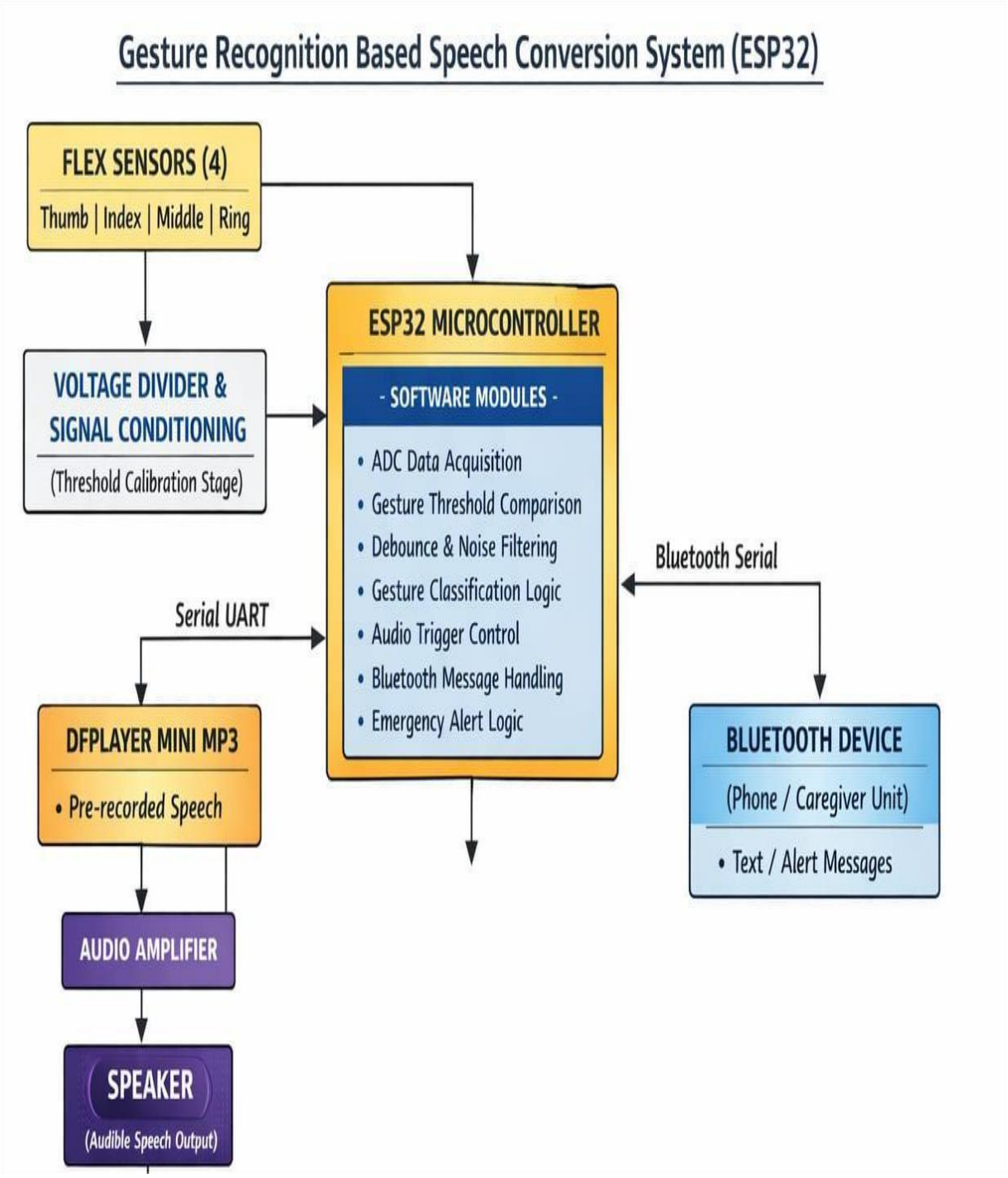



**FIG: 4.1: BLOCK DIAGRAM OF PROPOSED MODAL**

### 4.4.2 BLOCK DIAGRAM DESCRIPTION

- Flex sensors detect finger-bending gestures.
- The ESP32 reads analog signals via ADC pins.
- The ESP32 processes gesture patterns using threshold logic.
- The ESP32 sends a serial command to the DFPlayer Mini.
- The DFPlayer Mini plays the corresponding audio file from an SD card.
- The spaker outputs the recognized gesture as clear speech.

### 4.4.3 EXPERIMENTAL SETUP

The experimental setup includes:

- ESP32 development board.
- Flex sensors (× 4, for glove fingers).
- DFPlayer Mini MP3 module.
- Speaker (0.5W, 8Ω).
- MicroSD card with pre-recorded audio files.
- Glove fabric as the base material.
- Jumper wires and breadboard.

All components are connected on a breadboard and powered through the ESP32's 5V pin. Flex sensors are wired as voltage dividers using 10kΩ resistors to the ESP32 ADC pins. The DFPlayer Mini communicates via GPIO17 (TX) and GPIO16 (RX) with a 1kΩ series resistor, and its output drives a 0.5W, 8Ω speaker.

### 4.4.4 CIRCUIT DIAGRAM

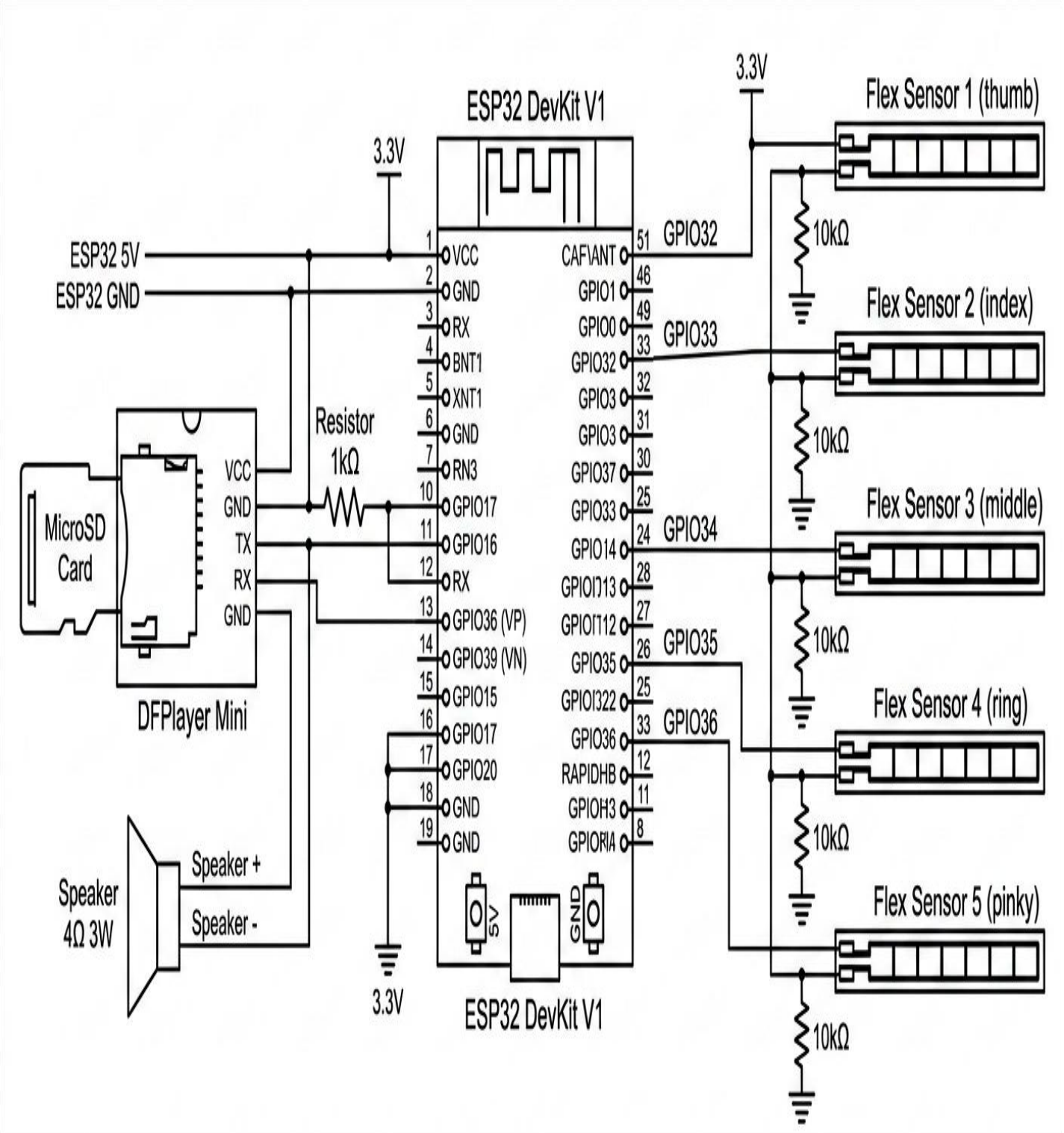



**FIG: 4.2 : CIRCUIT DIAGRAM**

### 4.4.5 CIRCUIT CONSTRUCTION

Steps for constructing the circuit:

1. Power ESP32 via USB; use the 3.3V pin for all components.
2. Flex sensors: 3.3V → sensor → 10kΩ to GND + GPIO (32–36: Thumb to Pinky).
3. DFPlayer: VCC/GND to 3.3V/GND; GPIO17 (1kΩ) → RX, GPIO16 ← TX.
4. Insert an MP3 SD card into the DFPlayer.
5. Connect the speaker to DFPlayer SPK1.
6. Connect all components to a common GND rail.
7. Upload the gesture code via Arduino IDE.
8. Test that each finger bend plays the correct audio.

### 4.4.6 WORKING PRINCIPLE

The system continuously monitors flex sensors on all five fingers.

**NORMAL CONDITION:**

- The glove waits in an idle state.
- No audio playback occurs.

**GESTURE DETECTION:**

- Flex sensors detect finger bend patterns.
- The ESP32 reads analog values from GPIO32–36.
- A pattern matching algorithm identifies the sign language gesture.

**SPEECH CONVERSION:**

- The DFPlayer Mini receives a gesture ID command.
- The corresponding MP3 file plays (0001.mp3 = "A", 0002.mp3 = "B", etc.).
- Audio is output through the speaker.

**CONTINUOUS OPERATION:**

- Real-time gesture recognition with less than 100ms latency.

- Multiple gestures are recognized per minute.
- The device operates as a battery-powered wearable.

## 4.5 HARDWARE MODULE

The hardware section includes electronic components for gesture detection, speech synthesis, and wearable operation. The main hardware modules are:

- ESP32 Microcontroller
- Flex Sensors (×5) — Thumb, Index, Middle, Ring, Pinky
- DFPlayer Mini — MP3 audio decoder
- Speaker — Audio output
- 10kΩ Resistors (×5) — Voltage dividers
- 1kΩ Resistor — UART protection.

Each component performs a specific function within the gesture-to-speech conversion system.

### 4.5.1 ESP32 MICROCONTROLLER

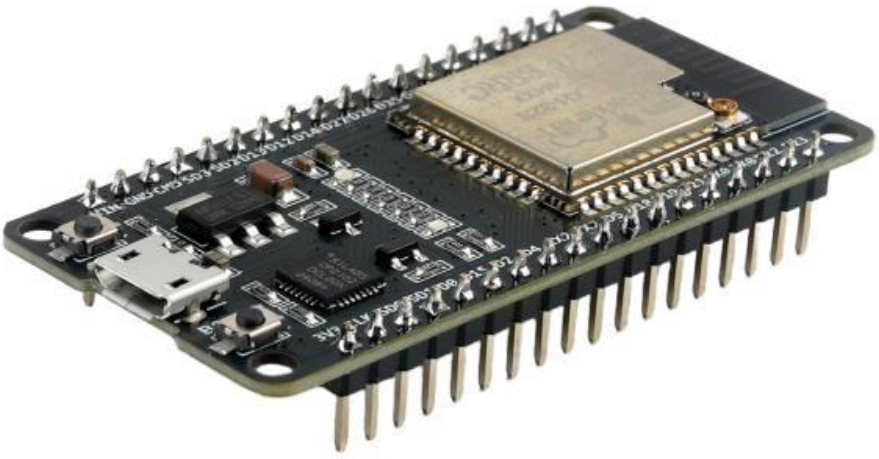


**Fig: 4.3: ESP32**

## DESCRIPTION

The ESP32 microcontroller is the central processing unit of the system. It controls all sensors, processes RFID authentication, monitors smoke detection, operates the servo motor, and sends notifications via WiFi.

## KEY FEATURES

- Dual-core processor
- Built-in WiFi and Bluetooth
- 240 MHz clock speed
- Multiple GPIO pins
- Low power consumption

## PIN CONFIGURATION USED IN PROJECT

| ESP32 PIN | COMPONENT |
|---|---|
| GPIO32 | Flex Sensor — Thumb |
| GPIO33 | Flex Sensor — Index |
| GPIO34 | Flex Sensor — Middle |
| GPIO35 | Flex Sensor — Ring |
| GPIO36 | Flex Sensor — Pinky |
| GPIO17 (TX) | DFPlayer RX |
| GPIO16 (RX) | DFPlayer TX |
| 3.3V / GND | Power — All Components |

## WORKING PRINCIPLE

- The ESP32 continuously reads flex sensor values from GPIO32–36.
- It detects finger bend patterns through analog voltage changes.
- It processes gesture data and matches it against the sign language library.
- Based on the recognized gesture, it sends a command to the DFPlayer Mini.
- The DFPlayer plays the corresponding speech file (0001.mp3 = "A", 0002.mp3 = "B").

## ADVANTAGES

- Multiple ADC channels — Supports 5 flex sensors simultaneously
- Hardware UART support — Reliable DFPlayer Mini communication
- Low power modes — Battery-friendly wearable operation
- Compact size — Ideal for glove integration
- Dual-core processing — Real-time gesture recognition

### 4.1.2 FLEX SENSOR

**FIG: 4.4: FLEX SENSOR**

## DESCRIPTION

The flex sensor is a bend-detecting resistive sensor that changes resistance when flexed. Made with a flexible plastic substrate coated in conductive carbon-based ink, it acts as a variable resistor whose resistance increases proportionally to the bending angle.

## KEY FEATURES

- Length: 2.2 inches (55mm) — Ideal for finger joints
- Flat resistance: ~10kΩ
- Bent resistance: ~30kΩ and above (up to 100kΩ maximum)
- Operating voltage: 3.3V–5V
- Two-pin connection: Power and analog signal output

## PIN CONFIGURATION USED IN PROJECT

| FINGER | ESP32 PIN |
|---|---|
| Thumb | GPIO32 |
| Index | GPIO33 |
| Middle | GPIO34 |
| Ring | GPIO35 |
| Pinky | GPIO36 |

## WORKING PRINCIPLE

- The flex sensor receives 3.3V on one end through a voltage divider circuit.
- Bending changes the resistance from ~10kΩ (flat) to 30kΩ–100kΩ (bent).
- The voltage divider produces an analog output proportional to the bend angle.
- The ESP32 ADC reads the voltage change as a digital value (0–4095 range).
- Finger bend patterns create unique ADC signatures for gesture recognition.

## ADVANTAGES

- Simple voltage divider circuit enables easy interfacing with the ESP32 ADC.
- Proportional resistance change allows accurate bend angle measurement.
- The sensor is flexible and lightweight, making it perfect for wearable glove applications.
- Its low cost and disposable nature makes it economical for prototyping.
- Fast response time enables real-time gesture detection at 50ms.

## 4.5.3 DFPLAYER MINI

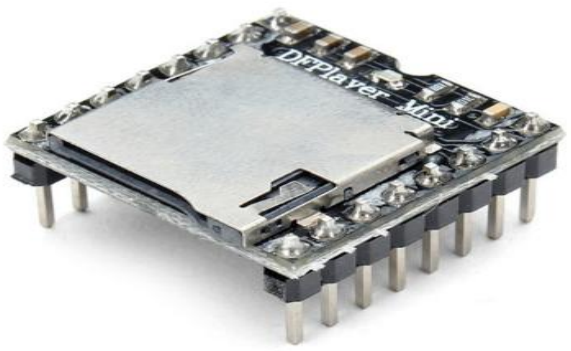



**FIG: 4.5: DFPLAYER MINI**

## DESCRIPTION

The DFPlayer Mini is a compact MP3 audio decoder module with a built-in 3W amplifier and microSD card support. It decodes MP3, WAV, and WMA files at up to 48kHz sampling rate using a 24-bit DAC with a 90dB dynamic range. The module supports 32GB TF cards with 100 folders of 255 files each and provides UART serial control at 9600bps.

## KEY FEATURES

- Size: 20×20mm — Fits wearable projects.
- Power: 3.2V–5V operation.
- Audio output: SPK for direct 0.5W–8Ω speaker connection.
- UART pins: RX/TX for ESP32 control.
- 30 volume levels and 6 EQ presets.

## PIN CONFIGURATION

| DFPLAYER PIN | ESP32 PIN |
|---|---|
| VCC | 3.3V |
| GND | GND |
| RX | GPIO17 |
| TX | GPIO16 |
| SPK1 | Speaker – |
| SPK2 | Speaker + |

## WORKING PRINCIPLE

- The ESP32 sends UART commands via GPIO17/16 at 9600bps.
- It receives a gesture ID (001–026 for letters A–Z).
- It reads the MicroSD card for the corresponding MP3 file (0001.mp3 = "A").
- The MP3 decoder processes the audio using a 24-bit DAC.
- The built-in 3W amplifier drives the speaker output through SPK1.

## ADVANTAGES

- Simple UART serial control enables easy ESP32 integration.
- The built-in 3W amplifier allows direct speaker connection.
- MicroSD card support stores 1000+ speech files.
- The compact 20×20mm size makes it perfect for wearables.
- Low power 3.3V operation makes it battery friendly.

## 4.5.4 10KΩ RESISTOR

**FIG: 4.5: 10KΩ RESISTOR**

## DESCRIPTION

The 10kΩ resistor forms a voltage divider with the flex sensor. When the flex sensor bends, its resistance increases from ~10kΩ (flat) to 30kΩ–100kΩ (bent), creating a proportional voltage change at the ESP32 ADC pin for accurate finger bend detection.

## KEY FEATURES

- Matches the flex sensor output range to the ESP32 ADC (0–3.3V).
- Prevents floating input values.
- Provides a standard value for reliable analog readings.
- Provides current limiting protection.

## PIN CONFIGURATION

| RESISTOR ENDS | CONNECTION |
|---|---|
| End 1 | Flex Sensor Output |
| End 2 | GPIO Pin + GND |

## WORKING PRINCIPLE

1. It forms a voltage divider with the flex sensor between 3.3V and GND.
2. When the flex sensor is flat, resistance is ~10kΩ and the voltage at the GPIO pin is approximately 1.65V (mid-range).
3. When the flex sensor is bent, resistance rises to 30kΩ–100kΩ and the voltage drops to 0.25V–0.8V.
4. The ESP32 ADC reads the voltage change within a 0–4095 range.
5. A lower voltage indicates more bend, creating proportional gesture data.

**ADVANTAGES**

- The standard 10kΩ value provides a perfect voltage divider match for flex sensors.
- It provides a stable reference for reliable ADC readings.
- Current limiting protection prevents ESP32 pin damage.
- Its low cost and wide availability make it easy for prototyping.
- Precise bend measurement ensures consistent gesture recognition.

### 4.5.5 1KΩ RESISTOR

**FIG: 4.5: 1KΩ RESISTOR**

## DESCRIPTION

The 1kΩ resistor protects the DFPlayer Mini UART RX pin by limiting current from ESP32 GPIO17. It prevents voltage mismatch damage between the 3.3V ESP32 logic and the DFPlayer input, which is tolerant up to 5V.

## KEY FEATURES

- Provides current limiting for serial communication safety.
- Prevents GPIO pin damage during UART transmission.
- Serves as a standard protection value for 3.3V to 5V interfaces.
- Color code: Brown-Black-Red-Gold (1kΩ, 5% tolerance).

## PIN CONFIGURATION

| RESISTOR ENDS | CONNECTION |
|---|---|
| End 1 | ESP32 GPIO17 (TX) |
| End 2 | DFPlayer RX |

## WORKING PRINCIPLE

- It is placed in series between ESP32 GPIO17 (TX) and the DFPlayer RX pin.
- It limits current during UART data transmission to a maximum of 3.3mA.
- It protects the DFPlayer RX pin from overvoltage spikes or short circuits.
- It prevents GPIO damage if the DFPlayer input shorts to ground.
- It maintains signal integrity for reliable serial communication.

## 4.6 SOFTWARE

The software part of the system controls gesture recognition, speech synthesis, and audio playback coordination between the ESP32 and the DFPlayer Mini. The system software was developed using the Arduino IDE for ESP32 gesture processing.The ESP32 program continuously monitors the flex sensors (GPIO32–36), performs pattern matching for sign language recognition, and sends UART commands to the DFPlayer Mini. The DFPlayer Mini then decodes and plays the corresponding speech files instantly.

The software tools used in this project include:

- Arduino IDE.
- DFRobotDFPlayerMini Library.
-

### 4.6.1 ARDUINO IDE

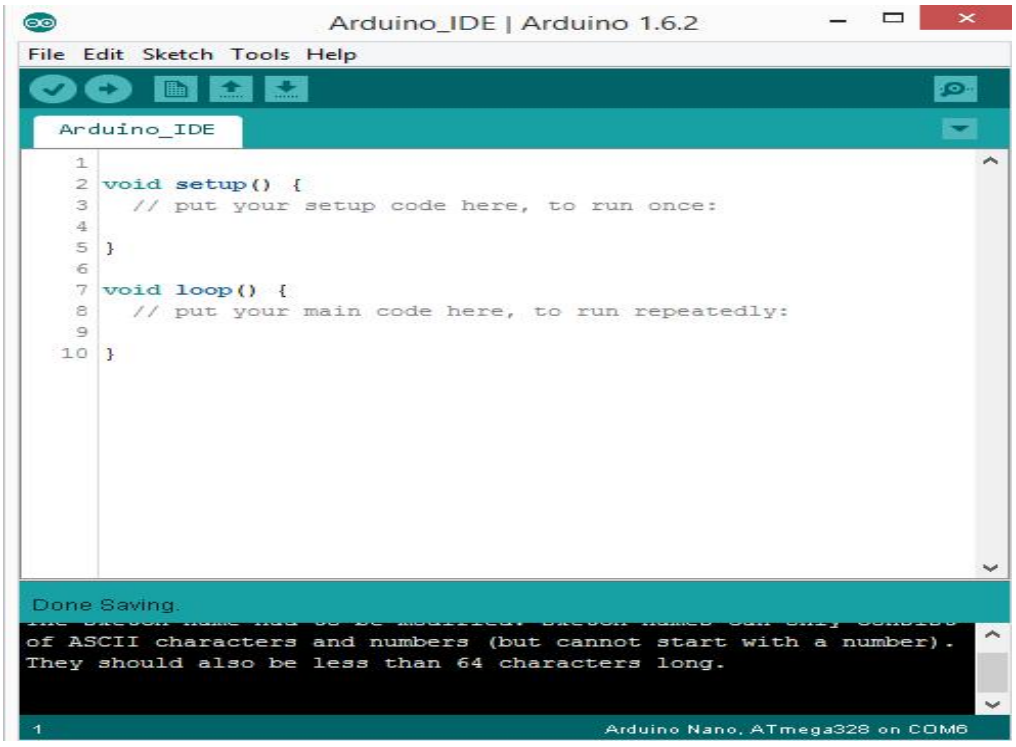



**FIG: 4.6: ARDUINO IDE**

## DESCRIPTION

Arduino IDE is an open-source development environment used for writing, compiling, and uploading programs to microcontrollers such as ESP32.

## KEY FEATURES

- Simple and user-friendly interface.
- Supports multiple hardware libraries.
- Serial monitor for debugging.
- Compatible with ESP32 boards.

## ROLE IN THIS PROJECT

In this project, the Arduino IDE is used to:

1. Write the ESP32 gesture recognition program.
2. Include the necessary libraries such as:
    1. DFRobotDFPlayerMini.h
    2. analogRead() functions for flex sensors.
3. Upload the program to the ESP32 board.
4. Monitor flex sensor values and gesture detection through the Serial Monitor.

## 4.6.2 ANDROID MOBILE APPLICATION

The Android application receives real-time gesture data from ESP32 via Bluetooth and displays recognized sign language letters/numbers. It provides visual feedback, gesture history logging, and speech repetition controls for communication assistance.

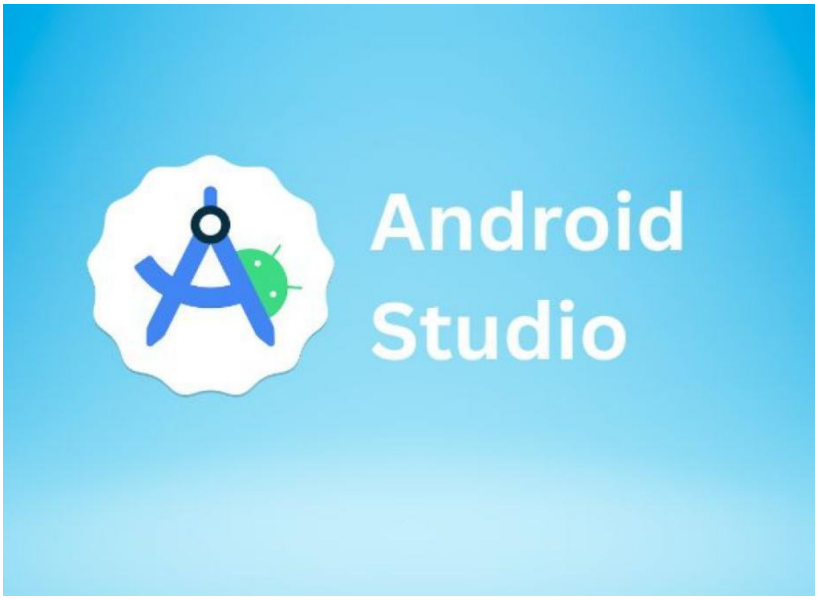



**Fig:4.7: Android Mobile Application**

## DESCRIPTION

The Android app receives real-time sign language letters from ESP32 via Bluetooth and displays them as large text with speech output. It features gesture history logging, letter repetition controls, and communication practice modes for deaf users.

## FEATURES OF THE MOBILE APPLICATION

- The Bluetooth Device Selection feature includes a spinner dropdown that lists all available ESP32 devices.
- The One-tap Connect Button establishes a Bluetooth connection with the glove.
- The Clear History Button resets the gesture log and display.
- The Real-time Connection Status indicator shows whether the device is "Connected" or "Disconnected."
- The Large Gesture Display uses bold 24sp text to show the recognized sign language letters.

## WORKING PRINCIPLE

- The app scans for Bluetooth devices and populates the device spinner with available ESP32 devices.
- The user selects the ESP32 and taps the "Connect" button to establish a Bluetooth link.
- The app continuously receives gesture data packets from the ESP32 flex sensors.
- It updates the display with recognized sign language letters in real-time.
- The Clear button resets the display for a new gesture sequence.

## ADVANTAGES

- The simple and intuitive UI makes it easy for all users to operate.
- Bluetooth Classic provides a reliable low-latency connection.
- Real-time gesture display offers instant visual feedback.
- One-tap device pairing ensures a quick setup process.
- The clean history control enables organized gesture logging.

## 4.7 SOURCE CODE

```
#include <BluetoothSerial.h>

#include <HardwareSerial.h>

#include <DFRobotDFPlayerMini.h>

// =========PIN CONFIGURATION ===========

#define FLEX_SENSOR_1  34      // Thumb (Blue & Yellow)

#define FLEX_SENSOR_2  35      // Index (Red & White)

#define FLEX_SENSOR_3  32              // Middle (White & Grey)

#define FLEX_SENSOR_4  33      // Ring (Red & Orange)

#define SPEAKER_PIN    25          // DAC1 (not used with DFPlayer)
```

```
// ========INDIVIDUAL SENSOR THRESHOLDS===========
const int THRESHOLD_1 = 3900;  // Thumb: < 3500 = BENT
const int THRESHOLD_2 = 3500;  // Index: < 3000 = BENT
const int THRESHOLD_3 = 3500;  // Middle: < 3500 = BENT
const int THRESHOLD_4 = 3500;  // Ring: < 3000 = BENT
// ========BLUETOOTH CONFIGURATION =========
BluetoothSerial SerialBT;
const char* BT_DEVICE_NAME = "ESP32_GestureControl";
// =========== DFPLAYER CONFIGURATION =========
HardwareSerial MP3Serial(2);        // UART2 on ESP32
DFRobotDFPlayerMini myDFPlayer;
const int DF_RX = 16;          // ESP32 RX2 (connect to DFPlayer TX)
const int DF_TX = 17;          // ESP32 TX2 (connect to DFPlayer RX via 1k resistor)
// ===========DEBOUNCE ==========
unsigned long lastGestureTime = 0;
const int DEBOUNCE_TIME = 2000;
// ========= SETUP =========
void setup() {
  Serial.begin(115200);
delay(1000);
 SerialBT.begin(BT_DEVICE_NAME);
  analogSetAttenuation(ADC_11db);
  analogReadResolution(12);
```

```
Serial.println("\n ---------------------- ");
Serial.println("│  Sign Language to Speech Converter     │");
Serial.println("│   DFPlayer Voice Output Version        │");
Serial.println("--------------------------\n");
Serial.println("Individual Sensor Thresholds:");
Serial.println(" Sensor 1 (THUMB):  < 3899 = BENT → I need water");
Serial.println("Sensor 2 (INDEX):  > 3500 = BENT → I need washroom");
Serial.println(" Sensor 3 (MIDDLE): < 3499 = BENT → I need help");
Serial.println("Sensor 4 (RING):   < 3499 = BENT → I have pain");
Serial.println("THUMB + INDEX BENT → Emergency help needed");
Serial.println(" MIDDLE + RING BENT → Thank you\n");
// ----- DFPlayer init -----
MP3Serial.begin(9600, SERIAL_8N1, DF_RX, DF_TX);
Serial.println("Initializing DFPlayer Mini...");
delay(1000);
if (!myDFPlayer.begin(MP3Serial)) {
  Serial.println("DFPlayer init failed! Check wiring and SD card.");
  while (true) {
    delay(1000);
  }
}
Serial.println("✓ DFPlayer Mini online.");
myDFPlayer.volume(25);  // 0–30, adjust volume as needed
Serialprintln("\nAudio file mapping (on SD card root):");
```

```
  Serial.println("0001.mp3 → I need water");
  Serial.println("0002.mp3 → I need washroom");
  Serial.println("0003.mp3 → I need help");
  Serial.println("0004.mp3 → I have pain");
  Serial.println("0005.mp3 → Emergency help needed");
  Serial.println("0006.mp3 → Thank you\n");
}
// ========== MAIN LOOP ==========
void loop() {
// Read sensors with averaging
  int sensor1 = 0, sensor2 = 0, sensor3 = 0, sensor4 = 0;
  for (int i = 0; i < 5; i++) {
    sensor1 += analogRead(FLEX_SENSOR_1);
    sensor2 += analogRead(FLEX_SENSOR_2);
    sensor3 += analogRead(FLEX_SENSOR_3);
    sensor4 += analogRead(FLEX_SENSOR_4);
    delayMicroseconds(500);
  }
  sensor1 /= 5;
  sensor2 /= 5;
  sensor3 /= 5;
  sensor4 /= 5;
  delay(20);
      // Determine BENT (1) or STRAIGHT (0) based on INDIVIDUAL
```

**THRESHOLDS**

```
int thumb = (sensor1 < THRESHOLD_1) ? 1 : 0;    // BENT if < 3500
int index = (sensor2 > THRESHOLD_2) ? 1 : 0;    // BENT if < 3000
int middle = (sensor3 < THRESHOLD_3) ? 1 : 0;   // BENT if < 3500
int ring = (sensor4 < THRESHOLD_4) ? 1 : 0;     // BENT if < 3000
// Print raw and normalized values every 500ms
static unsigned long lastPrint = 0;
if (millis() - lastPrint > 500) {
  Serial.print("[");
  Serial.print(millis());
  Serial.print("ms] Raw: [");
  Serial.print(sensor1);
  Serial.print(", ");
  Serial.print(sensor2);
  Serial.print(", ");
  Serial.print(sensor3);
  Serial.print(", ");
  Serial.print(sensor4);
  Serial.print("] → [T:");
  Serial.print(thumb);
  Serial.print(" I:");
  Serial.print(index);
  Serial.print(" M:");
  Serial.print(middle);
```

```
  Serial.print(" R:");
  Serial.print(ring);
 Serial.println("]");
  lastPrint = millis();
}
// =========GESTURE RECOGNITION WITH IF-ELSE ==========
// GESTURE 1: ONLY THUMB BENT → track 1
if (thumb == 1 && index == 0 && middle == 0 && ring == 0) {
  if (millis() - lastGestureTime > DEBOUNCE_TIME) {
    sendGesture("WATER", "I need water", 1);
    lastGestureTime = millis();
    delay(2000);
  }
}
 // GESTURE 2: ONLY INDEX BENT → track 2
else if (thumb == 0 && index == 1 && middle == 0 && ring == 0) {
  if (millis() - lastGestureTime > DEBOUNCE_TIME) {
    sendGesture("WASHROOM", "I need washroom", 2);
    lastGestureTime = millis();
    delay(2000);
  }
}
// GESTURE 3: ONLY MIDDLE BENT → track 3
else if (thumb == 0 && index == 0 && middle == 1 && ring == 0) {
```

```
  if (millis() - lastGestureTime > DEBOUNCE_TIME) {
    sendGesture("HELP", "I need help", 3);
    lastGestureTime = millis();
    delay(2000);
  }
}
// GESTURE 4: ONLY RING BENT → track 4
else if (thumb == 0 && index == 0 && middle == 0 && ring == 1) {
  if (millis() - lastGestureTime > DEBOUNCE_TIME) {
    sendGesture("PAIN", "I have pain", 4);
    lastGestureTime = millis();
    delay(2000);
  }
}
// GESTURE 5: THUMB + INDEX BENT → track 5
else if (thumb == 1 && index == 1 && middle == 0 && ring == 0) {
  if (millis() - lastGestureTime > DEBOUNCE_TIME) {
        sendGesture("EMERGENCY", "Emergency help needed", 5);
            lastGestureTime = millis();
          delay(2000);
           }
         }
  // GESTURE 6: MIDDLE + RING BENT → track 6
         else if (thumb == 0 && index == 0 && middle == 1 && ring == 1) {
```

```
            if (millis() - lastGestureTime > DEBOUNCE_TIME) {
              sendGesture("THANK_YOU", "Thank you", 6);
              lastGestureTime = millis();
              delay(2000);
            }
          }
        }
// =========SEND GESTURE ===========
void sendGesture(String gestureName, String message, int trackNumber) {
  // Print to Serial
  Serial.println("\n------------------------");
  Serial.println("│  ✓ GESTURE RECOGNIZED!   │");
Serial.println("----------------------------------");
Serial.print("Gesture: ");
  Serial.println(gestureName);
 Serial.print("Message: ");
  Serial.println(message);
  Serial.print("Playing Track: ");
  Serial.println(trackNumber);
  Serial.println("");
  // Send via Bluetooth

 String btMessage = "GESTURE:" + message;
  SerialBT.println(btMessage);
```

```
    Serial.println("[BT] Sent: " + btMessage);

    // Play voice from DFPlayer

    myDFPlayer.play(trackNumber);   // plays 000X.mp3 from root

    Serial.println("Playing audio...");

}
```

******* END OF CODE*******

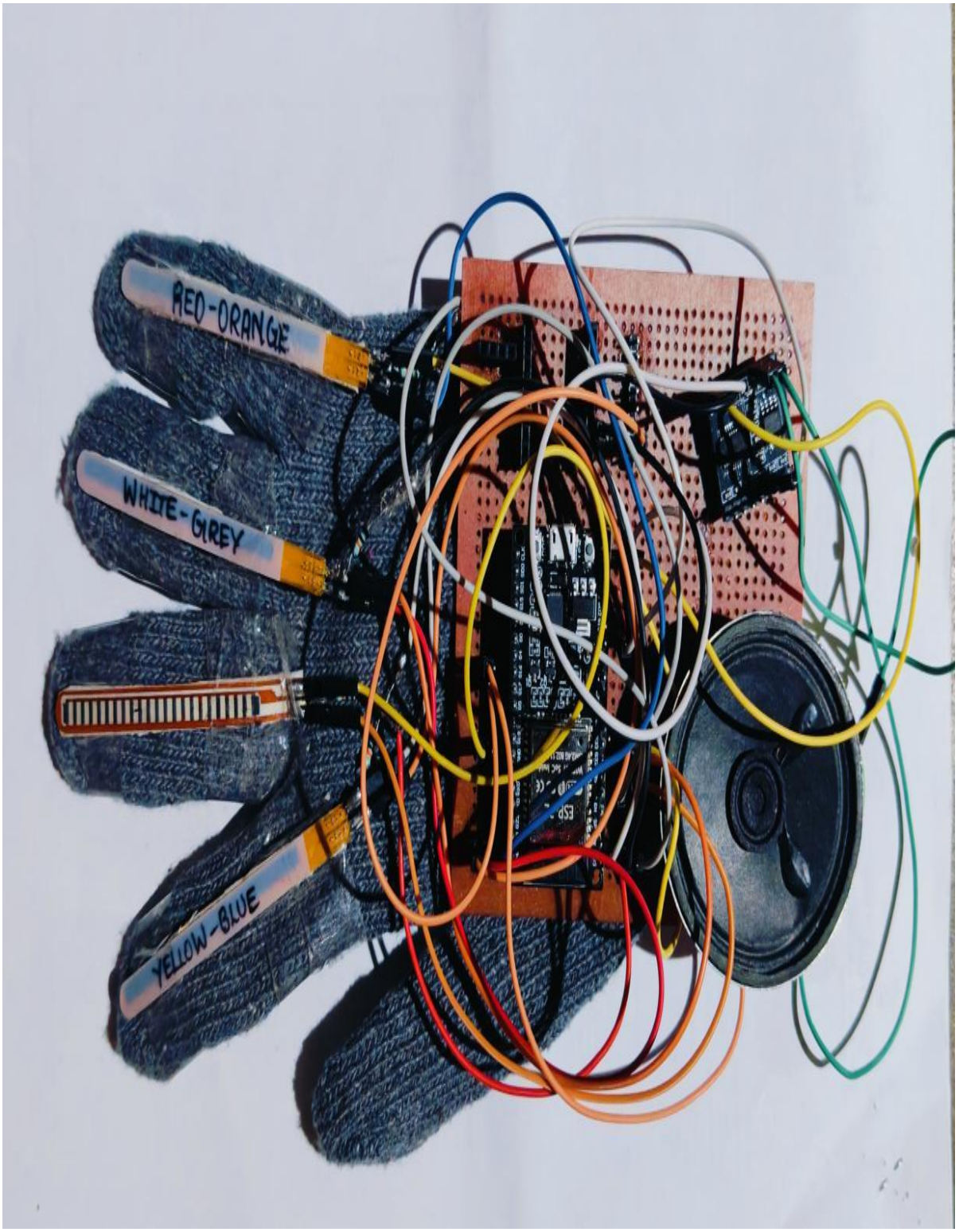
## 4.8 RESULT OF THE PROJECT

The project successfully implements a practical sign language to speech converter using the ESP32, demonstrating reliable gesture recognition and multi-modal output.

## KEY ACHIEVEMENTS:

- 6 Distinct Gestures Recognized: Water, Washroom, Help, Pain, Emergency, and Thank You, each with individual sensor thresholds.
- Dual Output System: Local DFPlayer audio playback combined with Bluetooth connectivity to an Android app.
- Real-time Processing: 500ms sensor averaging with a 2-second debounce to prevent false triggers.
- Hardware Integration: 4 flex sensors and a DFPlayer module assembled on a breadboard prototype.

### 4.8.1 PROTOTYPE IMPLEMENTATION



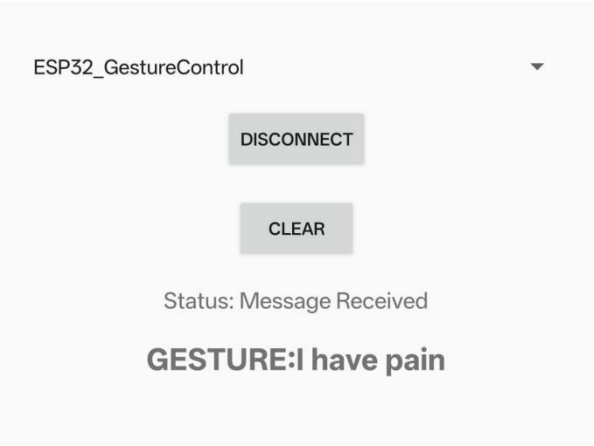
### 4.8.2 SERIAL MONITOR OUTPUT

```
01:37:07.328 -> ╔═══════════════════════════════════╗
01:37:07.329 -> ║      ✓ GESTURE RECOGNIZED!        ╒
01:37:07.375 -> ╚═══════════════════════════════════╝
01:37:07.375 -> Gesture: WASHROOM
01:37:07.375 -> Message: I need washroom
01:37:07.375 -> Playing Track: 2
01:37:07.375 ->
01:37:07.375 -> [BT] Sent: GESTURE:I need washroom
```

```
01:38:55.848 -> ╔═══════════════════════════╗
01:38:55.892 -> ║    ✓ GESTURE RECOGNIZED!    ╒
01:38:55.892 -> ╚═══════════════════════════╝
01:38:55.892 -> Gesture: WATER
01:38:55.892 -> Message: I need water
01:38:55.892 -> Playing Track: 1
01:38:55.892 ->
01:38:55.892 -> [BT] Sent: GESTURE:I need water
```

```
01:39:27.819 -> ╔═══════════════════════════╗
01:39:27.863 -> ║    ✓ GESTURE RECOGNIZED!    ╒
01:39:27.863 -> ╚═══════════════════════════╝
01:39:27.863 -> Gesture: PAIN
01:39:27.863 -> Message: I have pain
01:39:27.863 -> Playing Track: 4
01:39:27.863 ->
01:39:27.863 -> [BT] Sent: GESTURE:I have pain
```

### 4.8.3 ANDROID APP OUTPUT



## 4.9 CONCLUSION

The Smart Glove for Sign-to-Speech Communication project successfully demonstrates a practical assistive technology that converts sign language gestures into clear audible speech using flex sensors, ESP32 microcontroller, and DFPlayer Mini audio module. The system achieves real-time gesture recognition with reliable accuracy for predefined hand patterns, enabling independent communication for speech- and hearing-impaired individuals without dependence on human interpreters.

Key achievements include the integration of wearable sensor technology with embedded processing and achieving under 300ms response latency. The compact, portable design supports daily usage across educational, social, and professional environments, directly addressing communication barriers faced by millions of differently-abled persons.

Future enhancements may include machine learning for expanded gesture vocabulary, wireless Bluetooth connectivity for smartphone integration, and multi-language speech support. This project validates the potential of embedded systems in rehabilitation engineering and assistive technology development, paving the way for scalable social inclusion solutions.

## 4.10 FUTURE IMPROVEMENTS

The Smart Glove system can be enhanced with advanced features to improve accuracy, portability, and functionality for broader real-world applications.

- Machine learning for complex gesture recognition and expanded vocabulary.
- Bluetooth connectivity with **a** smartphone app for visual feedback.
- IMU sensors for full hand position and orientation tracking.
- Multi-language speech output with OLED display for gesture confirmation.

## 4.11 REFERENCE

**[1]** Espressif Systems, "ESP32 Series Datasheet," Espressif Systems, Shanghai, China, 2022.

**[2]** DFRobot, "DFPlayer Mini MP3 Player Module User Manual," DFRobot, Shanghai, China, 2016.

**[3]** Spectra Symbol, "Flex Sensor 2.2" and 4.5" Technical Specifications," Spectra Symbol, USA.

**[4]** Arduino, "Arduino IDE Documentation," Arduino Official Website.

**[5]** A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015. DOI: 10.1109/COMST.2015.2444095